

chita（猎豹）自建架构 · 详细版

单机终端 Coding Agent · Harness Layer · 评估驱动 · 供应链可审计 · 2026-08-09 · 133/133 测试

1. 总览

chita 是单机终端 coding agent：单二进制（Bun compile --compile），OpenAI 兼容 provider，评估驱动改进，tape-first 持久化。定位 Harness Layer——冻结模型、重写外围 harness，让 eval 数据驱动 harness 自身改进。技术栈：TypeScript + Bun（bun:sqlite / bun:test / bun build），vendor pi-tui 提供 TUI 组件。

- 会话可恢复：tape-first 存储 + compaction，崩溃/换终端不丢上下文
- 评估驱动：先写 eval 再开发，traces 锚定改进（8/8 eval 真实 DeepSeek 可复现 PASS）
- 供应链可审计：text lockfile、bun audit、MCP/Skills pinning

2. 包架构与依赖

包	职责	关键导出
cli	入口/配置/setup/banner	Config, loadConfig, apiKey, runSetup, renderBanner, VERSION
agent	执行核心（13 模块）	AgentLoop, ContextManager, runJudge, JudgeBudget, runSubagent, runWorkflow,
ai	Provider 抽象	OpenAICompatibleProvider, toOpenAIMessages
session	持久化	Tape, buildSessionTree, forkWithSummary, SqliteIndex, TraceEvent
tools	工具注册	ToolRegistry, registerBuiltinTools, ToolContext
tui	交互界面	startTui + vendor(pi-tui)

注意：packages/agent/src/index.ts 是 36 字节 stub（仅 `export const name`），不 re-export——消费方全部相对路径直接 import（如 TUI 用 ../../agent/src/loop.ts）。依赖方向：ai → agent（协议类型在 loop.ts）；agent → session/tools；cli/tui → 全部。

3. 核心循环 AgentLoop（agent/src/loop.ts）

状态机

IDLE → THINKING（每轮起点）→ TOOL_CALL（provider 流式 tool_call）→ OBSERVING（工具执行完）→ 回 THINKING 或 DONE。done 工具是唯一 DONE 硬闸门；AbortError → CANCELLED；overflow 超 3 次 / 其他错误 / 轮数或 token 耗尽 → ERROR。WAITING_USER 已声明未赋值（M1.5+ 预留）。

LoopOptions 全字段

字段	类型	含义
cwd	string	工作目录（进 ToolContext）
provider	Provider	LLM 抽象，chat() 流式返回 StreamEvent
tools?	ToolRegistry	缺省 registerBuiltinTools 自建
hooks?	LoopHooks	生命周期钩子（见下）
maxIterations?	number	默认 50
maxTokens?	number	全会话 token 上限，默认 1,000,000
autoApproveAsk?	boolean	--print 开发模式：ask 级工具自动批准
mode?	build plan	plan 只读（写工具拦截，bash 需显式批准）
signal?	AbortSignal	取消当前轮；setSignal() 可刷新（一次性 controller 会永久污染）

LoopHooks 全字段

钩子	签名	触发点
beforeToolCall	(name,args,ctx)=>Promise<boolean>	执行前权限/审计拦截，false 拒绝
afterToolCall	(name,result)=>result void	执行后改写/脱敏 (v2.1 F4)
onEvent	(event: TraceEvent)=>void	tool_call/tool_result/context_truncated/error
onAssistantMessage	(msg)=>void	流式 assistant 片段 (UI)
onSessionEnd	(state,summary?)=>void	DONE/ERROR/CANCELLED

公有方法：run(初始任务，重置会话) / continue(追加轮次，多轮续聊) / steer、followUp(轮间注入 system) / setSignal / getTokensUsed、restoreTokens / seedConversation(校验工具配对，孤立 tool 抛错) / getConversation。

4. 上下文管理 (context.ts + compact.ts)

- estimateTokens：gpt-tokenizer(cl100k) 可选加载，回退启发式 (CJK ≈ 1.5 tok/字符、ASCII ≈ 4 字符/tok，保守上界)
- ContextManager：threshold = $0.9 \times \text{maxTokens}$ ；truncate 从头丢，永远保留首条 user 任务；工具配对不变量（丢 tool 结果连同 assistant tool_call）
- compact：六段压缩——Goal / Constraints & Preferences / Progress / Key Decisions / Next Steps / Critical Context；保留 task + 最近至多 8 条；newest 不得以孤立 tool 开头；段 <2 条回退 truncate
- extractSummary：确定性提取器（无 LLM）——goal=首条 user 前 200 字符；decisions 匹配 (decided|决定|采纳|rejected)；next 匹配 (next|下一步|然后)；critical 正则抓文件路径/函数名/错误文本（ ≤ 30 项）
- overflowRecovery：>3 次溢出终止 (shouldTerminate)，防 API 空烧；compact 未实际减少也终止

5. 错误分层 (errors.ts)

类别	判定	可重试	退避
rate_limit	HTTP 429	是	1000ms
server	HTTP ≥ 500	是	2000ms
auth	HTTP 4xx	否	—
overflow	消息含 overflow/context length/token limit	是（走 compact 重试）	
timeout	含 timeout/network/econnreset	是	1500ms
malformed	含 malformed/invalid json	是	—
other	其余	否	—

backoffDelay：指数 $\min(\text{max}, \text{base} \times 2^{\text{attempt}}) \times \text{jitter} \pm 20\%$ ；默认策略 maxRetries=3。错误消息绝不原样回喂重试上下文 (includeErrorInContext=false)。

6. 持久化：tape 与 session 树 (session 包)

tape = append-only JSONL，单一事实源 (SQLite 仅索引)。事件逐行追加永不重写；fork 复制前缀到新文件（源不动）。

TraceEvent 8 类：message / tool_call / tool_result / judge / context_truncated / error / done / usage；基字段 seq/type/ts/tokens；SessionMeta{sessionId, cwd, model, provider, createdAt, parentId?, branchSummary?, pinnedResources?}。

session-tree：buildSessionTree 构建父子分支树；forkWithSummary 记录分支原因；mergeBranchBack 合并。SqliteIndex：bun:sqlite + FTS5 全文索引 messages，支持 search/recentSessions。Tape 方法：open/append/appendMeta/readAll/fork/readMeta/markCrashed/close（含 stale-lock 接管）。

7. 工具系统 (tools 包)

工具	参数	说明
read	path	读取文件 (UTF-8)
write	path, content	写文件, 自动建父目录
bash	command, timeout?	执行命令, 超时/截断
grep	pattern, path	正则搜索, 返回 文件:行号
ls	path	列目录
glob	pattern	通配匹配
git	args[]	只读子命令 (status/diff/log)
done	summary?	DONE 硬闸门 (唯一出口)

权限模型 allow/ask/deny : ToolRegistry 在 execute 前按 tool.defaultPermission + beforeToolCall 钩子裁决 ; ToolContext{cwd, permission, signal} 。 scrubSecrets (agent) 经 afterToolCall 接入 cli 与 tui, 工具输出进模型/tape 前脱敏。MCP (agent/src/mcp.ts) : @modelcontextprotocol/sdk studio 客户端, 远端工具包装为 <name>_<tool>, defaultPermission=ask, 30s 超时, 输出 4096 截断, 失败关闭已连句柄。

8. Provider 抽象 (ai 包)

OpenAICompatibleProvider.chat(messages, {signal, tools}) 是 SSE 流式 generator, 产出 StreamEvent : message (assistant 文本) / tool_call (含 args) / usage ({tokens,input,output}, 无条件上报) / done (无工具轮才发——工具轮由 loop 执行后回喂) 。 toOpenAIMessages 映射内部消息→OpenAI 线格式 (tool 消息带 tool_call_id) 。

9. 子代理与 workflow (subagent.ts + workflow.ts)

- runSubagent : 独立 AgentLoop 隔离 session ; 模型分层 main/cheap (主编排、便宜执行) ; permissionScope inherit→build 否则 plan ; maxIterations 默认 15
- TaskResult 证据契约 : state=DONE 且 summary 且 verificationHint 三缺一即 fail (防 10 步内过早终止)
- runWorkflow : 固定阶段序列 (非通用引擎) ; 每阶段 ≤maxRetries(2) 次重试, 每次新建 loop ; 指数退避封顶 5s ; 首败即停
- specPipeline 模板 : implement → verify → finalize

10. 记忆与技能 (memory.ts + skills.ts + sediment.ts)

- 四层记忆 : /.chita/{MEMORY.md, checkpoint.md, notes.md, tasks.md} ; injectMemory 预算注入, 优先级 tasks>checkpoint>notes>memory, 超预算层跳过
- RecurrenceGate : 重复触发门 (阈值 2) , 持久化 ~/.chita/stats/recurrence.json, 跨重启存活 ; 成本 -87%
- skills : 渐进式披露——indexSkills 只加载 name/description (不含正文) , loadSkill 按需读 SKILL.md ; 目录 ~/.chita/skills//SKILL.md (项目 .chita/skills/ 合并)
- sediment : 成功后从 trace 提 SKILL.md 草稿 (≥3 工具全成功) , PENDING 状态永不自动激活, 需人工 activateSkill

11. 独立验证 (judge.ts)

runJudge 用独立模型 (绝不用执行模型) 对 conversation 判 verdict : pass/fail/uncertain + reason + evidence 。 JudgeBudget 三闸 : 每会话 ≤3 次、每 10 轮节流、月度 \$10 (0.3\$/M token 计) 持久化 ~/.chita/stats/judge.json 。

12. CLI (cli 包)

入口	行为
chita (无参)	进 TUI (自动恢复最近会话)
chita "任务"	--print 模式跑任务
chita --plan	只读分析 (plan 模式)
chita --judge	运行 + /goal 独立验证
chita init	生成 ~/.chita/config.json
chita --version	ASCII 猎豹头 banner

Config 字段：provider / model / permissionDefault(allow|ask|deny) / maxTokensPerTask / contextWindow (白名单 CONFIG_KEYS，未知键永不进内存)；contextWindow 按模型名推断 (deepseek*→1M，-8k/-32k/-1m 后缀解析，claude→200K 等)。apiKey 优先级：环境变量 CHITA_API_KEY > ~/.chita/.env。setup 引导：5 provider 选择 + 掩码输入 key (支持退格/Ctrl+U/粘贴) + 可输入 h 导入 Hermes key + 写 ~/.chita/.env。

13. TUI (tui 包，vendor pi-tui)

VStack 布局：对话区 (Markdown，grow) → 工具条 (固定 5 行，独立滚动，40 条窗口化) → 输入行 → 状态栏。

- slash 命令：/help /new /tree /resume[无参=最近] /fork /mode build|plan /goal /tool /exit (quit/exit 无斜杠别名)
- 状态栏：session / mode / model / ↑input ↓output (K/M 单位) / ctx 当前会话长度/窗口 (百分比)
- Ctrl+C：kitty 键盘协议双路径 (\u0003 或 ESC[99;5u)，空闲单次退出、运行中首按取消次按退出
- 流式：onAssistantMessage 聚合渲染；endStreaming 一次性写完整 assistant 到 tape + usage 快照
- 启动：Text 组件 banner (猎豹头 ASCII，不经过 Markdown) + 自动恢复最近会话 (读 tape + restoreTokens)
- 工具条：tool_call 缓存命令，结果行显示 命令+首参+行数 (cat ~/.hermes/config.yaml · 114 lines)；banner echo 折叠为 XN
- vendor：@earendil-works/pi-tui pin commit 7bdb16c28d794a5ff8e7485479c8e37eccd9a8d8，2 本地 patch (Enter 提交、tableBorder 主题)，MIT

14. 供应链审计

- vendor pin commit + VENDOR.md 记录 (含本地 patch 说明，排除 native/)
- MCP/Skills pinning stub (name/version/commit/hash/source) 记录于 SessionMeta.pinnedResources
- API key 只在 ~/.chita/.env 或环境变量，永不进 config.json/对话/tape (scrub 兜底)
- bun audit + text lockfile (bun.lock)

15. 评估体系 (evals)

evals/cases 8 场景：e01 读项目 / e02 改文件 / e03 跑测试 / e04 修 bug / e05 git log / e06 grep 搜索 / e07 一致编辑 / e08 输出验证。真实 DeepSeek 可复现 PASS；每次改进由 trace 锚定。

16. 运行状态目录 (~/.chita/)

路径	内容
~/.chita/config.json	配置 (白名单键)
~/.chita/.env	API key (权限私有)

~/chita/agent/sessions/<cwd-key>/sess-*.json	tape 事件日志（append-only）
~/chita/skills/	技能（SKILL.md）
~/chita/skills-pending/	技能草稿（PENDING）
~/chita/stats/judge.json	judge 预算（月键）
~/chita/stats/recurrence.json	记忆重复门
<repo>/chita/{MEMORY,checkpoint,notes,logs,md	四层记忆

17. 测试与验收

133 测试 / 23 文件（bun:test），含 loop 状态机、context 压缩、tape 持久化、session-tree、judge 预算、tools 权限、banner 渲染、TUI 行为。每轮改动过 agent-bridge 审阅（Cursor 独立确认后交付）。